

CPP Module 06

Guía didáctica completa
Conversión de tipos en C++

ex00 ScalarConverter ex01 Serializer ex02 Identify

42 Network · Recopilatorio docente

Índice

1. Visión general del módulo y los 4 casts de C++
2. Ex00 — ScalarConverter (`static_cast`)
3. Ex01 — Serializer (`reinterpret_cast`)
4. Ex02 — Identify (`dynamic_cast`)
5. Profundización: `const` en `operator=`
6. Profundización: RTTI
7. Profundización: vtables y `vptr`
8. Profundización: alignment y padding
9. Checklist final de defensa
10. Recursos en vídeo

1 · Visión general

C++ tiene **cuatro tipos de cast** que reemplazan al cast de C (`(int)x`). El módulo 06 está diseñado para que entiendas *cuándo* y *por qué* usar cada uno.

Cast	Cuándo usarlo	Ejercicio
<code>static_cast<T></code>	Conversiones lógicas entre tipos relacionados (int ↔ float, derived → base)	ex00
<code>reinterpret_cast<T></code>	Reinterpretación de bits (puntero ↔ entero)	ex01
<code>dynamic_cast<T></code>	Downcasting seguro en runtime (Base* → Derived*)	ex02
<code>const_cast<T></code>	Quitar/añadir <code>const</code>	(no aparece)

Regla mental: cuanto más arriba en la tabla, más seguro. `reinterpret_cast` es "yo sé lo que hago, compilador, cállate".

2 · Ex00 — ScalarConverter

2.1 El problema

Recibes un literal como string ("42" , "42.0f" , "a" , "nan") y debes convertirlo a `char` , `int` , `float` y `double` .

2.2 Conceptos clave

`static_cast` vs cast de C

```
double d = 42.5;
int a = (int)d;           // C-style: funciona pero opaco
int b = static_cast<int>(d); // C++: explícito, buscable, verificable
```

`static_cast` falla en compile-time si la conversión no tiene sentido (`static_cast<int*>(d)` no compila). El cast de C lo aceptaría como `reinterpret_cast` . **Por eso 42 obliga a usar los casts C++.**

`std::numeric_limits<T>`

```
std::numeric_limits<int>::max()           // 2147483647
std::numeric_limits<float>::quiet_NaN()
std::numeric_limits<double>::infinity()
```

Detección de pseudo-literales

Subject exige manejar: `nan` , `nanf` , `+inf` , `-inf` , `+inff` , `-inff` . No son números: son strings que debes detectar antes de parsear.

Formato de salida exacto

- `float` siempre con sufijo `f`
- Si entero, añadir `.0` antes del sufijo
- Char no imprimible → `Non displayable` (mayúscula, sin guión)
- Fuera de rango o NaN/inf → `impossible` (minúscula)

2.3 FAQ y troubleshooting

FAQ 1: "¿Por qué `42.0f` me sale como `42.50f` ?"

```
std::cout << static_cast<float>(value) << ".0f";    // ❌
```

Concatena `.0f` siempre. Solución:

```
if (f == static_cast<long long>(f))
    std::cout << std::fixed << std::setprecision(1) << f << "f";
else
    std::cout << f << "f";
```

FAQ 2: "Mi overflow de int no se detecta para 2147483648"

`INT_MAX` al convertirse a `double` se redondea a `2147483648.0`. La comparación `value > INT_MAX` es falsa. Cast explícito a `double` y filtrar NaN/inf antes.

FAQ 3: "Enteros grandes salen en notación científica + .0"

`cout` usa notación científica con valores grandes (`2.14748e+09`) y al añadir `.0f` queda malformado. Solución: `std::fixed << std::setprecision(1)`.

🔥 Errores más comunes en 42

1. No detectar `42.0f` como input válido (validador rechaza la `f`).
2. Mensaje literal mal escrito: `"non-displayable"` en vez de `"Non displayable"`.
3. Casts directos sobre NaN/inf → UB.
4. Imprimir NaN con `cout << quiet_NaN()` : el formato depende de libc.
5. `operator=` declarado `const` (heredado de OCF mal entendida).

3 · Ex01 — Serializer

3.1 El problema

Convertir un puntero a struct en entero `uintptr_t` (serializar) y recuperarlo preservando:
`deserialize(serialize(p)) == p`.

3.2 Conceptos clave

¿Qué es `uintptr_t`?

Alias de `<stdint.h>` para un entero sin signo **garantizado lo bastante grande para almacenar cualquier puntero**. En sistemas 64-bit es `unsigned long`.

¿Por qué `reinterpret_cast`?

```
uintptr_t n = static_cast<uintptr_t>(ptr);    // ❌ compile error
uintptr_t n = reinterpret_cast<uintptr_t>(ptr); // ✅ reinterpreta bits
```

Mental model: `reinterpret_cast` no transforma datos; renombra cómo el compilador ve esos bytes.

Clase no instanciable

```
class Serializer {
private:
    Serializer();
    Serializer(const Serializer&);
    ~Serializer();
    Serializer& operator=(const Serializer&);
public:
    static uintptr_t serialize(Data* ptr);
    static Data*    deserialize(uintptr_t raw);
};
```

3.3 FAQ

FAQ: "¿Puedo usar `(uintptr_t)ptr` ?"

Funcionalmente sí, pero el cast C es ambiguo: el compilador prueba `const_cast` , `static_cast` y `reinterpret_cast` en orden. 42 quiere que demuestres que sabes elegir.

Errores más comunes en 42

1. `operator=` declarado `const` (mismo bug que ex00).
2. Demostrar la deserialización imprimiendo el struct original en vez de `recovered_ptr->campo` .
3. No entender por qué `uintptr_t` y no `int / long` .

4 · Ex02 — Identify

4.1 El problema

Jerarquía `Base ← A, B, C`. `generate()` crea aleatoriamente uno y devuelve `Base*`. Debes recuperar **en runtime** el tipo real.

4.2 Conceptos clave

Comportamiento de `dynamic_cast`

Caso	Éxito	Fallo
Puntero: <code>dynamic_cast<D*>(b)</code>	Puntero válido	<code>NULL</code>
Referencia: <code>dynamic_cast<D&>(b)</code>	Referencia válida	<code>std::bad_cast</code>

¿Por qué la diferencia? Una referencia **no puede ser NULL** en C++. La única forma de señalar fallo es excepción.

¿Por qué prohíben `<typeinfo>`?

Porque `typeid()` haría el ejercicio trivial:

```
if (typeid(*p) == typeid(A)) ... // 1 línea, sin aprender nada
```

42 quiere que **entiendas** `dynamic_cast`.

4.3 FAQ

FAQ 1: "Mi `dynamic_cast` no compila — 'Base' is not polymorphic"

Falta `virtual` en algún método de `Base`. El destructor virtual es la solución estándar.

FAQ 2: "identify(*NULL) me da segfault"

Desreferenciar NULL es UB
antes

de entrar en la función. El chequeo de NULL solo sirve en la versión que recibe puntero.

FAQ 3: "¿catch std::bad_cast o std::exception?"

`std::bad_cast` es más preciso pero requiere `<typeinfo>` (¡prohibido!). Captura `std::exception`.

Errores más comunes en 42

1. Olvidar `virtual` en destructor de `Base` → no compila o memory leak.
2. Hacer trampa en `identify(Base&)` usando `&p` internamente.
3. Incluir `<typeinfo>` "porque lo pide el catch" — fail automático.
4. Desreferenciar NULL en el test de "unknown".
5. No sembrar `srand()` → secuencia siempre igual.
6. `delete` doble o no llamarlo → leaks detectados por `valgrind`.

5 · Profundización: `const` en `operator=`

5.1 La regla básica

```
class Foo {  
    void bar() const;    // "este método NO modifica *this"  
};
```

Dentro del método, `this` se vuelve `const Foo*`. Modificar miembros falla en compile-time.

5.2 ¿Qué hace `operator=`?

Su propósito es modificar `*this` :

```
Foo& operator=(const Foo& other) {  
    this->x = other.x;    // ¡modifico el objeto!  
    return *this;  
}
```

Si lo declaras `const`, la asignación no compila. **Es contradicción semántica.**

5.3 Los tres lugares donde aparece `const`

Lugar	Ejemplo	Significado
Parámetro	<code>operator=(const Foo& other)</code>	No modifico <code>other</code>
Retorno	<code>const Foo& getFoo()</code>	Lo devuelto es inmutable
Method qualifier	<code>void print() const</code>	No modifico <code>*this</code>

5.4 OCF correcta

Método	<code>const</code> en parámetro	<code>const</code> al final
Constructor	✓ sí	✗ no aplica
Copy constructor	✓ sí	✗ no aplica
<code>operator=</code>	✓ sí	✗ NO
Getter	✓ sí	✓ sí
<code>print()</code>	—	✓ sí

5.5 ¿Por qué nuestro código compilaba con el bug?

Si nunca llamas al `operator=` (clase no instanciable como `ScalarConverter`), el compilador genera el método pero nadie lo invoca. En cuanto intentes `obj1 = obj2;`, no compila.

6 · Profundización: RTTI

6.1 ¿Qué es?

RTTI = Run-Time Type Information. Metadata embebida por el compilador para preguntar en runtime "¿qué tipo real es este objeto?".

Accesible vía:

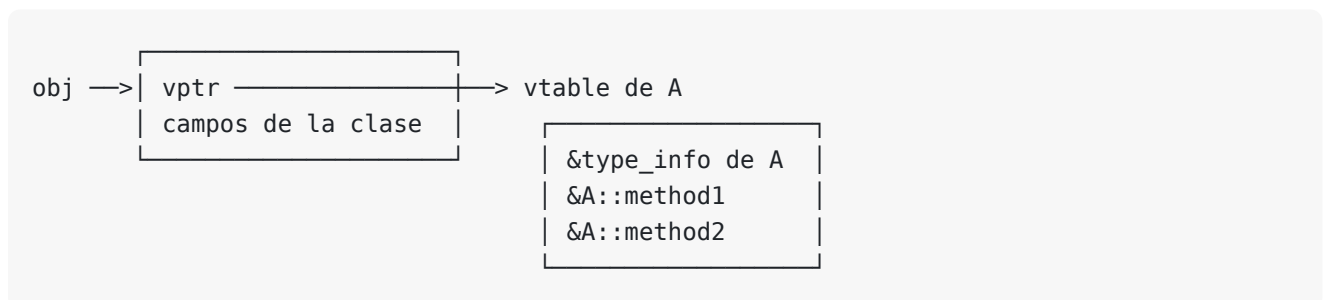
- `dynamic_cast<T>` (usado en ex02)
- `typeid()` (prohibido en ex02)

6.2 ¿Cuándo se activa?

Solo si la clase tiene al menos un método virtual → clase polimórfica. Sin polimorfismo el tipo se conoce en compile-time.

6.3 ¿Dónde vive?

En la **vtable**. Cada vtable contiene un puntero a un objeto `type_info` que describe la clase.



6.4 Coste

- **Espacio:** +1 puntero por objeto polimórfico (vptr, 8 B en 64-bit).
- **Tiempo:** `dynamic_cast` recorre la jerarquía → no es O(1) puro.

7 · Profundización: vtables y vptr

7.1 El problema que resuelven

```
class Animal { public: virtual void sound(); };
class Dog : public Animal { void sound(); };
class Cat : public Animal { void sound(); };

Animal* a = new Dog();
a->sound(); // ¿Cómo sabe llamar Dog::sound y no Animal::sound?
```

El compilador no sabe en compile-time qué objeto real apunta `a`. Necesita despachar en runtime → vtable.

7.2 Estructura

Una vtable por clase polimórfica (estática). Cada objeto tiene un puntero oculto (`vptr`) a su vtable.

vtable de Dog:

type_info de Dog
&Dog::sound
&Animal::breathe

vtable de Cat:

type_info de Cat
&Cat::sound
&Animal::breathe

7.3 Despacho de `a->sound()`

1. Cargar `vptr` desde el objeto.
2. Indexar la vtable en la posición de `sound`.
3. Llamar al puntero almacenado allí.

7.4 Footgun: virtual en el constructor

El `vptr` se asigna **en el constructor**. Llamar a un método virtual desde el constructor de `Base` NO llama al override de `Derived`:

```
class Base {
public:
    Base() { init(); }    // △ llama Base::init, NO Derived::init
    virtual void init();
};
```

7.5 ¿Por qué destructor virtual?

```
Base* p = new Derived();
delete p;    // ¿~Base() o ~Derived()?
```

- Sin `virtual` → solo `~Base()` . Si `Derived` tenía recursos → **leak**.
- Con `virtual` → busca en vtable → `~Derived()` , que encadena `~Base()` .

8 · Profundización: alignment y padding

8.1 ¿Qué es alignment?

La CPU exige que ciertos tipos vivan en direcciones múltiplos de su tamaño. Si no, lectura más lenta (x86) o crash (ARM antigua).

Tipo	Alignment x86-64
char	1
short	2
int, float	4
double, puntero, long	8

8.2 Padding silencioso

```
struct Bad {
    char a;    // offset 0, +7 padding
    double b;  // offset 8
    char c;    // offset 16, +7 padding (sizeof múltiplo de 8)
};
// sizeof(Bad) = 24 (no 10)

struct Good {
    double b;  // offset 0
    char a;    // offset 8
    char c;    // offset 9, +6 padding
};
// sizeof(Good) = 16
```

Regla pulgar: ordena campos de mayor a menor tamaño.

8.3 Relación con ex01

```
typedef struct {
    std::string first_name;    // 32 B
    std::string last_name;    // 32 B
    int age;                  // 4 + 4 padding
    float money;              // 4
    bool has_job;             // 1 + 7 padding
} Data;
```

Cuando `reinterpret_cast` -eas el puntero a entero y vuelves, **no copias bytes**: solo conservas la dirección. El alignment se preserva automáticamente.

8.4 Mini-experimento

```
class NoVirtual { int x; };
class WithVirtual { int x; public: virtual ~WithVirtual(); };
WithVirtual::~~WithVirtual() {}

// 64-bit:
// sizeof(NoVirtual)    = 4
// sizeof(WithVirtual) = 16  ← 8 vptr + 4 int + 4 padding
```

9 · Checklist final de defensa

- ☐ `ScalarConverter / Serializer` no instanciables (todo privado).
- ☐ `operator=` NO declarado `const`.
- ☐ `static` en todos los métodos públicos de utilidad.
- ☐ `Base` con destructor virtual público.
- ☐ No incluyes `<typeinfo>` en `ex02`.
- ☐ Compila con `-Wall -Wextra -Werror -std=c++98` sin warnings.
- ☐ `valgrind ./binary` → 0 leaks, 0 errors.
- ☐ Sabes explicar **por qué** cada cast es el adecuado.
- ☐ Mensajes exactos: `impossible`, `Non displayable`.
- ☐ Pseudo-literales: `nan`, `nanf`, `±inf`, `±inff`.

Preguntas frecuentes de defensa

1. ¿Por qué `static_cast` y no `(int)`? Verificación en compile-time; cast de C es ambiguo.
2. ¿`reinterpret_cast<int*>(0x42)`? Compila, pero acceder es UB.
3. ¿`dynamic_cast` sin virtuales? No: requiere RTTI → requiere polimorfismo.
4. ¿Por qué `uintptr_t` y no `long`? Portabilidad; `long` no garantiza tamaño de puntero.
5. Diferencia `dynamic_cast<T*>` vs `<T&>` al fallar. Puntero NULL; referencia `bad_cast`.

10 · Recursos en vídeo

Enlaces verificados al momento de la generación. Si alguno cae, busca el título en YouTube.

10.1 Casts en C++ (español)

- [Jesús Conde](#) — `static_cast`, `dynamic_cast` o funciones virtuales
- [Dibcode](#) — C++ Conversión de Tipos (`static_cast`)
- [Jesús Conde](#) — Uso de `dynamic_cast`

10.2 RTTI y vtables

- [The Cherno](#) — Virtual Functions in C++ (vtable mechanics, EN)
- [The True Boolean](#) — ¿Qué demonios es la VTable en C++? (ES)

10.3 Alignment y padding

- [DeepDiveDev](#) — Alineación de memoria en C/C++ (ES)
- [C++ Weekly Ep. 410](#) — Padding and Alignment (EN, Jason Turner)

10.4 Canales generales recomendados

- [MoureDev](#) — [Brais Moure](#) (POO, fundamentos, comunidad ES)
 - [MoureDev TV](#) (formato corto)
 - [The Cherno](#) (serie C++ completa, EN)
 - [C++ Weekly with Jason Turner](#) (EN, deep dives)
-